

Software Design Document

SmartClass LMS

Final Report

Instructor : Yasin Sazid & Proma Chowdhury

Lecturer, Department of CSE, East West University

Course Title : Software Engineering

Course Code: CSE412

Section : 05

Submitted by

MD. Afridi

ID: 2022-3-60-080

Date of Submission: 20.08.26

TABLE OF CONTENTS

Chapter 1 Introduction	1
1.1 Project Overview	1
1.2 Objectives and Scope	1
1.3 Stakeholders	3
1.4 Technology Stack	3
Chapter 2 Software Requirements Specification & Analysis	5
2.1 Stakeholder Needs & Analysis	5
2.2 List of Requirements	6
2.3 Quality Function Deployment	10
2.4 Requirements Modeling	12
Chapter 3 Software Design	18
3.1 Architectural Design	18
3.2 Component-Level Design	21
3.3 User Interface Design	23
Chapter 4 Implementation	26
4.1 Code Structure Overview	26
4.2 GitHub Repository	27
Chapter 5 Deployment	28
5.1 Deployment Platform	28
5.2 Deployment Process	28
5.3 Website URL	28
Chapter 6 Conclusion	29
6.1 Key Learnings	29
6.2 Limitations	29
6.3 Future Plan	30

Chapter 1. Introduction

1.1 Project Overview: User Story

As a Teacher, I want a centralized digital platform where I can create and manage classrooms, distribute learning materials, assign coursework, conduct live quizzes, record attendance using secure QR codes, communicate with students, and monitor academic performance through real-time analytics, **so that** I can reduce administrative workload, improve classroom engagement, and dedicate more time to teaching rather than paperwork.

As a Student, I want to join classes using a simple class code, access study materials anytime, submit assignments online or through offline verification when internet connectivity is limited, participate in quizzes, receive instant grades and feedback, monitor my academic progress through analytics, and interact with an AI learning assistant, **so that** I can learn efficiently, stay organized, and improve my academic performance regardless of location or internet availability.

As a Guardian (Parent), I want to securely connect my account with my child's academic profile, monitor attendance, grades, assignment completion, quiz performance, and teacher feedback, receive timely notifications about important academic events, and download comprehensive progress reports, **so that** I can actively support my child's education and remain informed without needing frequent visits to the educational institution.

1.2 Objectives and Scope

1.2.1 Project Objectives

1. **Centralise classroom management:** Provide a single platform that handles class creation, student enrolment, announcements, and content distribution for all active classes simultaneously.
2. **Automate administrative overhead:** Replace manual paper-based attendance with a QR code session system that records student check-ins in under five seconds.

3. **Enable comprehensive assessment:** Provide teachers with tools for creating assignments, MCQ quizzes, and live polls with automatic grading and instant result delivery to students.
4. **Deliver data-driven analytics:** Compute per-student performance metrics — average grade, attendance rate, quiz average, and submission rate — and present them as interactive charts using Chart.js, accessible to teachers, students, and guardians.
5. **Support digital equity:** Allow students with unreliable internet connectivity to generate offline QR submission tokens that a teacher can verify in person, ensuring no student is penalised for connectivity limitations.
6. **Provide AI-powered academic assistance:** Integrate a context-aware AI chatbot that is aware of each user's role, enrolled classes, grade averages, and attendance rate, enabling personalised academic guidance around the clock.
7. **Enable guardian oversight:** Allow parents and guardians to link to their child's account with student approval, and access a dedicated monitoring portal with attendance records, grades, and downloadable progress reports.

1.2.2 Scope

SmartClass LMS is a self-hosted, server-side rendered (SSR) web application. The following table defines the scope boundaries of the system.

Table 1.1: Technology Stack

Feature Domain	Description
Multi-Role Access	Role-based dashboards for Teachers, Students, and Guardians
Classroom Management	Digital class creation, joining, announcements, and stream feed
Assessment System	Assignments, grading, MCQ quizzes, polls with auto-scoring
Attendance Tracking	QR-code session-based attendance and manual override
Analytics Engine	Per-student performance dashboards with Chart.js charts
Communication	Class chat, direct messages, in-app notifications
AI Chatbot	Context-aware academic assistant via OpenRouter AI API

Guardian Portal	Parent access to student grades, attendance, and reports
Offline Support	QR-token-based submission for connectivity-limited students
Content Management	Material upload, bulk transfer across classes, personal archive
Scheduling	Calendar, video meet sessions via external link integration

Out of Scope: Payment gateway integration, native mobile application, automated grading of subjective written answers, and real-time video streaming.

1.3 Stakeholders

The following stakeholders have direct or indirect interests in the SmartClass LMS system.

Table 1.2: Stakeholder Summary

Stakeholder	Type	Role in System
Teacher / Instructor	Primary	Creates and manages classes, posts content, grades submissions
Student / Learner	Primary	Joins classes, submits assignments, takes quizzes
Guardian / Parent	Primary	Monitors student academic performance
System Administrator	Secondary	Manages server, database, and user accounts
School / Institution	Secondary	Deploys and funds the system; sets academic policies
OpenRouter AI API	External System	Provides AI inference for the chatbot feature
QR Server API	External System	Generates QR code images for attendance sessions

1.4 Technology Stack

The system employs a lightweight, dependency-minimal technology stack chosen for ease of deployment on a standard XAMPP environment, maintainability without a build pipeline, and strong security through server-side rendering.

Table 1.1: Technology Stack

Layer / Component	Technology
-------------------	------------

Backend	PHP 8.x (Procedural + PDO Data Objects)
Database	MySQL 8.x (utf8mb4_unicode_ci, 22 relational tables)
Frontend	HTML5, CSS3, JavaScript ES6+
Styling	Custom CSS with CSS Variables (Dark Theme System)
Icons	Font Awesome 6.5 (via CDN)
Fonts	Google Fonts — Inter (via CDN)
Web Server	Apache 2.4 (XAMPP Environment, Windows)
AI Integration	OpenRouter AI API — NVIDIA Nemotron Model (free tier)
QR Code Generation	QR Server API (api.qrserver.com, external HTTP)
Charts	Chart.js 4.x (via CDN — bar, doughnut, line charts)
Video Conferencing	External link integration (Google Meet, Zoom)
Version Control	Git / GitHub

The deliberate decision to avoid frontend frameworks such as React or Vue.js ensures that the application loads rapidly on low-specification hardware and slow network connections — a critical consideration for educational deployments in resource-constrained environments. Core functionality remains fully operational without JavaScript, with JavaScript providing progressive enhancements such as real-time polling, modal overlays, and Chart.js visualisations.

Chapter 2. Software Requirements Specification & Analysis

2.1 Stakeholder Needs & Analysis

2.1.1 Primary and Secondary Stakeholders

Stakeholder identification and analysis was conducted during the requirements elicitation phase. Stakeholders were categorised as primary (direct system users whose needs directly shape functional requirements) and secondary (those with indirect interests in system reliability, security, or cost-effectiveness).

Table 2.1: Stakeholder Analysis Needs

Stakeholder	Type	Needs
Teacher	Primary	Reduce administrative overhead; centralise class resources
Student	Primary	Easy access to class materials, assignments, and feedback
Guardian	Primary	Transparent view of child's academic progress
Institution	Secondary	System reliability, data privacy, cost-effectiveness
System Admin	Secondary	Ease of deployment, database maintainability
AI API Provider	External	Correct API usage, rate limits respected

2.1.2 Methods for Requirement Elicitation

The following elicitation techniques were systematically applied during the requirements gathering phase to ensure a comprehensive understanding of stakeholder needs.

Table 2.2: Requirement Elicitation Methods

Method	Description	Stakeholders Involved	Outcome
User Story Mapping	Personas for Teacher, Student, and Guardian written to capture daily workflows	All primary stakeholders	Core feature list per role

Prototype Walkthroughs	Early UI wireframes reviewed by prospective users for feedback	Teachers, Students	Sidebar layout, modal-based forms, dark-theme preference
Document Analysis	Existing manual attendance sheets, assignment trackers, and report cards studied	Teachers, Institution	Database schema for submissions, attendance, performance_cache
Observation	A typical university classroom session observed to understand interaction patterns	Teachers, Students	Led to QR attendance feature requirement
Informal Interviews	Discussions with students about online learning pain points	Students	Offline submission (QR token) and AI chatbot requirements
Surveys / Questionnaires	Questions about preferred communication methods within a classroom	Students, Teachers	Class chat, direct messages, and notification preferences

2.2 List of Requirements

2.2.1 Functional Requirements (FRs)

FR-01: User Authentication & Profile Management

ID	Requirement
FR-01.1	The system SHALL allow users to register with name, email, password, and role (Teacher / Student / Guardian).
FR-01.2	The system SHALL authenticate users via email and BCrypt-hashed password comparison.
FR-01.3	The system SHALL maintain session state for authenticated users using PHP Sessions.
FR-01.4	The system SHALL allow users to update their profile (name, bio, phone, avatar photo).
FR-01.5	The system SHALL redirect users to a role-specific dashboard upon successful login.
FR-01.6	The system SHALL allow users to log out, destroying their session securely.

FR-02: Classroom Management

ID	Requirement
FR-02.1	The system SHALL allow Teachers to create a class with name, section, subject, room, description, colour theme, and optional logo.
FR-02.2	The system SHALL generate a unique 7-character alphanumeric class code upon class creation.
FR-02.3	The system SHALL allow Students to join a class by entering a valid class code.
FR-02.4	The system SHALL enforce a maximum student enrolment limit (default: 40 students per class).
FR-02.5	The system SHALL allow Teachers to post announcements in the class stream with optional file attachments.
FR-02.6	The system SHALL allow Teachers to broadcast announcements and materials to all their active classes simultaneously.
FR-02.7	The system SHALL allow Teachers to archive a class, placing it in a 48-hour grace period before permanent deletion.
FR-02.8	The system SHALL allow Teachers to restore an archived class within the 48-hour grace period.

FR-03: Assignment & Grading System

ID	Requirement
FR-03.1	The system SHALL allow Teachers to create assignments with title, description, due date, point value, and optional file attachment.
FR-03.2	The system SHALL allow Students to submit assignments via file upload or text entry.
FR-03.3	The system SHALL automatically detect and flag late submissions (submitted after due_date).
FR-03.4	The system SHALL allow one submission per student per assignment (prevent duplicates).
FR-03.5	The system SHALL allow Teachers to grade submissions with a numeric grade (0 to points) and written feedback.
FR-03.6	The system SHALL notify both Teacher (on new submission) and Student (on grading) via in-app notification.
FR-03.7	The system SHALL display submission status badges: submitted / graded / late / missing.

FR-04: QR-Code Attendance System

ID	Requirement
FR-04.1	The system SHALL allow Teachers to generate a timed QR attendance session for a class date.
FR-04.2	The system SHALL create a unique cryptographic token (64-character hex via bin2hex(random_bytes(32))) per session.
FR-04.3	The system SHALL generate a QR code image via external API encoding the scan URL and token.
FR-04.4	The system SHALL allow Students to scan the QR code and automatically mark their attendance as 'present'.
FR-04.5	The system SHALL reject duplicate attendance attempts (one check-in per student per date).
FR-04.6	The system SHALL expire QR sessions after the Teacher-defined duration (1 to 60 minutes).
FR-04.7	The system SHALL allow Teachers to manually mark attendance (present / absent / late / excused).
FR-04.8	The system SHALL display a live attendance list showing which students have checked in, refreshing every 3 seconds.

FR-05: Quiz & Poll System

ID	Requirement
FR-05.1	The system SHALL allow Teachers to create quizzes with title, time limit, and MCQ questions.
FR-05.2	The system SHALL allow Teachers to set quizzes to 'draft', 'live', or 'closed' status.
FR-05.3	The system SHALL display a countdown timer to Students while a live quiz is active.
FR-05.4	The system SHALL auto-grade quiz responses upon submission and return score and percentage.
FR-05.5	The system SHALL allow one quiz attempt per Student (prevent re-submissions).
FR-05.6	The system SHALL allow Teachers to create polls and students to vote on active polls (one vote per poll).
FR-05.7	The system SHALL display live poll results as vote percentages.

FR-06: Analytics & Performance Tracking

ID	Requirement
FR-06.1	The system SHALL compute per-student analytics including average grade, attendance rate (%), quiz average, and submission rate.
FR-06.2	The system SHALL cache computed analytics in the performance_cache table and refresh upon data change.
FR-06.3	The system SHALL render performance data as interactive charts (bar, line, doughnut) using Chart.js.
FR-06.4	The system SHALL allow Teachers to view analytics for any student in their classes.
FR-06.5	The system SHALL allow Guardians to view linked student analytics.

FR-07: Guardian Portal

ID	Requirement
FR-07.1	The system SHALL allow Guardians to send a link request to a Student by email address.
FR-07.2	The system SHALL require Student approval before the Guardian can view their data.
FR-07.3	The system SHALL allow Guardians to view linked students' grades, attendance records, and quiz scores.
FR-07.4	The system SHALL allow Guardians to download a progress report for linked students.

FR-08: Communication Features

ID	Requirement
FR-08.1	The system SHALL provide a class chat for all class members.
FR-08.2	The system SHALL support direct private messaging between users.
FR-08.3	The system SHALL show in-app notifications with unread count badge.
FR-08.4	The system SHALL support scheduling and joining video meet sessions via external links.

FR-09: AI Chatbot

ID	Requirement
FR-09.1	The system SHALL provide an AI academic assistant integrated into all user dashboards.
FR-09.2	The system SHALL build a role-aware context string and inject it into each AI prompt.
FR-09.3	The system SHALL maintain conversation history (last 10 exchanges) per user session.
FR-09.4	The system SHALL communicate with the OpenRouter AI API using the OpenAI-compatible format.

FR-10: Offline Submission

ID	Requirement
FR-10.1	The system SHALL allow Students to generate an offline QR token for a specific assignment when internet is unavailable.
FR-10.2	The system SHALL allow Teachers to scan the student's offline QR token to verify and record their submission.
FR-10.3	The system SHALL track offline token status: pending / submitted / verified.

2.2.2 Non-Functional Requirements (NFRs)

Non-functional requirements define the quality attributes of the system — how it behaves rather than what it does. These requirements are enforced through architectural decisions and coding standards applied consistently throughout the codebase.

Table 2.13: Non-Functional Requirements

ID	Category	Requirement
NFR-01.1	Security	All passwords SHALL be stored using BCrypt hashing (password_hash() with PASSWORD_DEFAULT).
NFR-01.2	Security	All database queries SHALL use PDO prepared statements to prevent SQL injection.
NFR-01.3	Security	All user-generated content rendered to HTML SHALL be escaped via htmlspecialchars() to prevent XSS.
NFR-01.4	Security	Every API endpoint SHALL validate \$_SESSION['user_id'] and the user's role before execution.
NFR-02.1	Performance	Dashboard page load time SHALL be under 2 seconds on a local XAMPP server with up to 100 enrolled students.
NFR-02.2	Performance	Analytics computation SHALL use the performance_cache table to avoid redundant aggregate queries.
NFR-02.3	Performance	QR attendance polling (every 3 seconds) SHALL use a lightweight JSON endpoint executing fewer than 5 database queries.
NFR-03.1	Usability	The UI SHALL follow a consistent dark-theme design system using CSS variables across all pages.
NFR-03.2	Usability	All user feedback SHALL use toast notifications auto-dismissing within 3 seconds.
NFR-03.3	Usability	All creation and editing forms SHALL appear in modal overlays to avoid context loss.
NFR-04.1	Reliability	The system SHALL handle all API errors gracefully with user-friendly messages.
NFR-04.2	Reliability	The system SHALL prevent data loss by validating all required fields before database writes.
NFR-05.1	Maintainability	The shared config/db.php file SHALL be the sole point of database connection configuration.
NFR-06.1	Portability	The system SHALL run on any XAMPP/LAMP stack (PHP 7.4+ / MySQL 8.x).
NFR-06.2	Portability	The UI SHALL be responsive and functional on screen widths from 360px (mobile) to 1920px (desktop).

2.2.3 Extra-Ordinary Requirements (Wow Factors)

The following features exceed standard LMS expectations and differentiate SmartClass LMS from conventional platforms such as Google Classroom or Moodle.

Table 2.14: Extraordinary Requirements (Wow Factors)

ID	Feature	Description
----	---------	-------------

EX-01	AI-Powered Academic Chatbot	An embedded, context-aware AI assistant that knows the student's enrolled classes, average grade, and attendance rate — providing personalised academic guidance via the OpenRouter API.
EX-02	QR-Code Attendance System	Teacher generates a time-limited QR code; students scan with any camera-enabled device to instantly mark themselves present — no app installation required.
EX-03	Offline Submission via QR Token	Students without internet can generate a unique QR token for their assignment. The teacher later scans it in person to record the submission.
EX-04	48-Hour Archival Grace Period	Archived classes are not immediately deleted; teachers have a 48-hour window to restore them, preventing accidental data loss.
EX-05	Performance Analytics Dashboard	Multi-metric charts (grade trend, attendance rate, quiz performance) powered by Chart.js, accessible to teachers and guardians alike.
EX-06	Guardian Academic Monitoring Portal	Parents can link to their child's account with student approval and access a dedicated portal for grades, attendance, and downloadable reports.
EX-07	Cross-Class Material Broadcast	Teachers can broadcast study materials from one class to multiple classes simultaneously — a single-click operation.
EX-08	Live Interactive Polls	Teachers create real-time polls during class; students vote and results update live with percentage bars — enabling immediate engagement measurement.

2.3 Quality Function Deployment

2.3.1 Customer Requirements (CRs)

Table 2.15: Customer Requirements (CRs) — QFD Input

ID	Customer Requirement	Source Stakeholder	Priority
CR1	Teachers need a fast and simple way to take attendance	Teacher	High
CR2	Students want to submit assignments easily, even without internet	Student	High
CR3	Guardians want real-time visibility into their child's performance	Guardian	High
CR4	Teachers want instant quiz grading without manual effort	Teacher	High
CR5	Students want personalised academic help available 24/7	Student	Medium
CR6	The system must protect student data from unauthorised access	Institution	High
CR7	The interface must work on mobile phones and tablets	All Users	Medium
CR8	Teachers want to broadcast study materials to multiple classes at once	Teacher	Medium
CR9	Students want clear feedback on their academic progress	Student	High
CR10	The system should notify users about important events in real time	All Users	Medium

2.3.2 Engineering Requirements (TRs)

Table 2.16: Engineering / Technical Requirements (TRs)

ID	Technical Requirement	Addresses CR
TR1	Implement QR-code session generation with cryptographic token (64-char hex, bin2hex(random_bytes(32)))	CR1
TR2	Implement offline QR token system with pending/submitted/verified lifecycle	CR2
TR3	Implement Guardian Link workflow with student approval and progress report generation	CR3
TR4	Implement auto-grading engine for MCQ / True-False quiz questions with JSON answer comparison	CR4
TR5	Integrate OpenRouter AI API with role-aware context injection and 10-exchange session history	CR5
TR6	Implement BCrypt password hashing, PDO prepared statements, session-based RBAC, and XSS escaping	CR6
TR7	Implement responsive CSS with mobile breakpoints; sidebar collapses to hamburger on screens < 768px	CR7
TR8	Implement material transfer API (api/material_transfer.php) supporting bulk broadcast across teacher's classes	CR8
TR9	Implement analytics engine with Chart.js visualisations for grade, attendance, and quiz trends; cache in performance_cache	CR9
TR10	Implement in-app notification system with unread badge count; push alerts triggered by all major events	CR10

2.3.3 Quality Function Deployment Matrix (House of Quality)

The House of Quality matrix below maps Customer Requirements (CRs) to Engineering / Technical Requirements (TRs), showing the relationship strength using the standard QFD notation: Strong Relationship , Medium Relationship , Weak Relationship , and No Relationship . The Technical Priority Score is calculated as the sum of relationship weights for each TR column.

Table 2.17: House of Quality Matrix — SmartClass LMS QFD

CR	TR1 QR Att.	TR2 Offline	TR3 Guardi an	TR4 Auto-G rade	TR5 AI Chat	TR6 Securit y	TR7 Respon sive	TR8 Materi al	TR9 Analyti cs	TR10 Notifs
CR1 Fast Attenda nce	Strong	None	None	None	None	Med	Med	None	None	Weak
CR2 Offline Submit	Weak	Strong	None	None	None	Med	Med	None	None	Med
CR3 Guardia n	None	None	Strong	None	None	Strong	Med	None	Strong	Med

Visibility										
CR4 Quiz Grading	None	None	None	Strong	None	Med	None	None	Med	Med
CR5 24/7 AI Help	None	None	None	None	Strong	Med	Weak	None	None	None
CR6 Data Protection	None	Med	Med	None	None	Strong	None	None	None	None
CR7 Mobile / Tablet	Weak	Weak	Weak	None	Weak	None	Strong	None	Med	Med
CR8 Bulk Broadcast	None	None	None	None	None	None	None	Strong	None	Med
CR9 Progress Feedback	None	None	Med	Med	Med	None	Med	None	Strong	Strong
CR10 Notifications	Med	Weak	None	Med	None	None	Med	Med	None	Strong

Highest Priority Technical Requirements based on QFD score: TR6 — Security (33), TR9 — Analytics Engine (32), TR1 — QR Attendance (29), TR10 — Notification System (27), TR3 — Guardian Portal (26). These represent the critical engineering investments that deliver the greatest satisfaction across the broadest set of customer requirements.

2.4 Requirements Modeling

2.4.1 Use Case Diagrams

Use case diagrams model the functional interactions between actors and the system. Three role-specific use case diagrams are provided, each capturing the full set of use cases accessible to that role. The system-level diagram provides an overview of all 34 identified use cases distributed across eight functional subsystems.

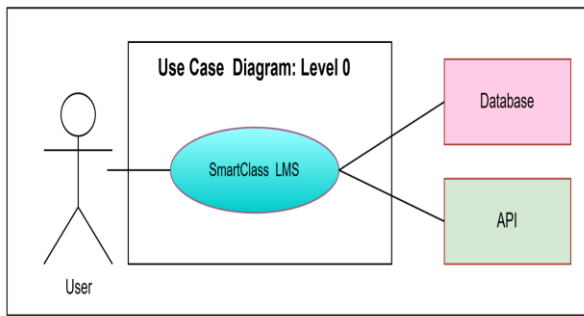


Figure 2.1 : Use Case Diagram for Level 0

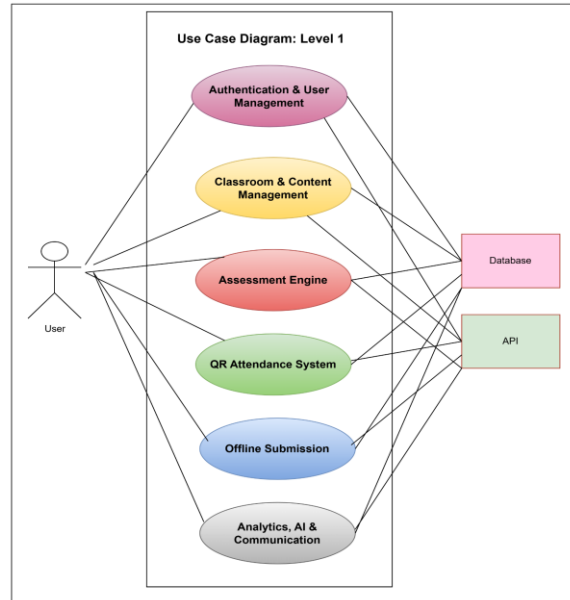


Figure 2.2 : Use Case Diagram for Level 1

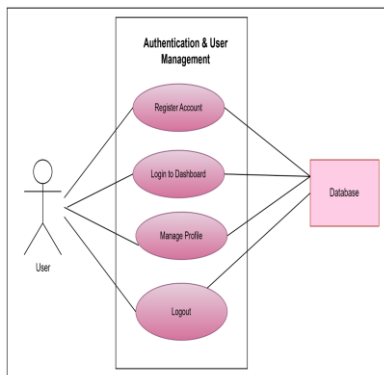


Figure 2.3 : Use Case Diagram for Authentication & Register

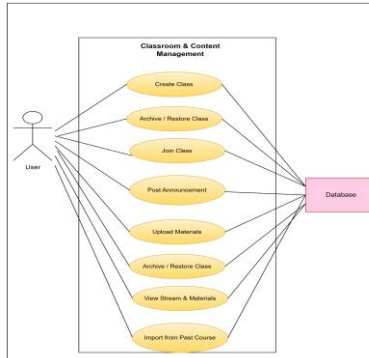
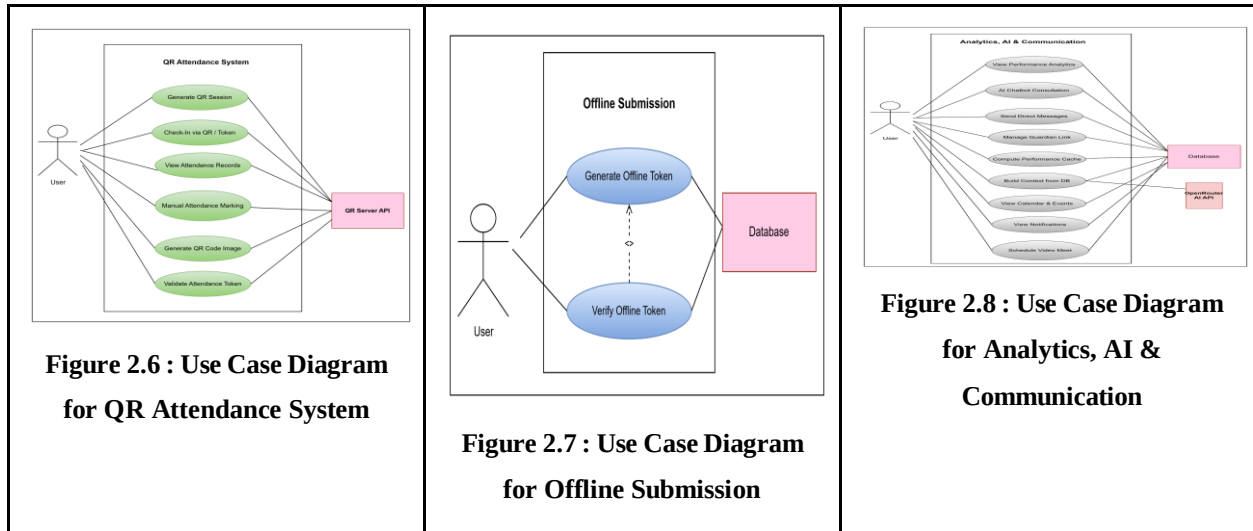


Figure 2.4 : Use Case Diagram for Classroom & Content Management

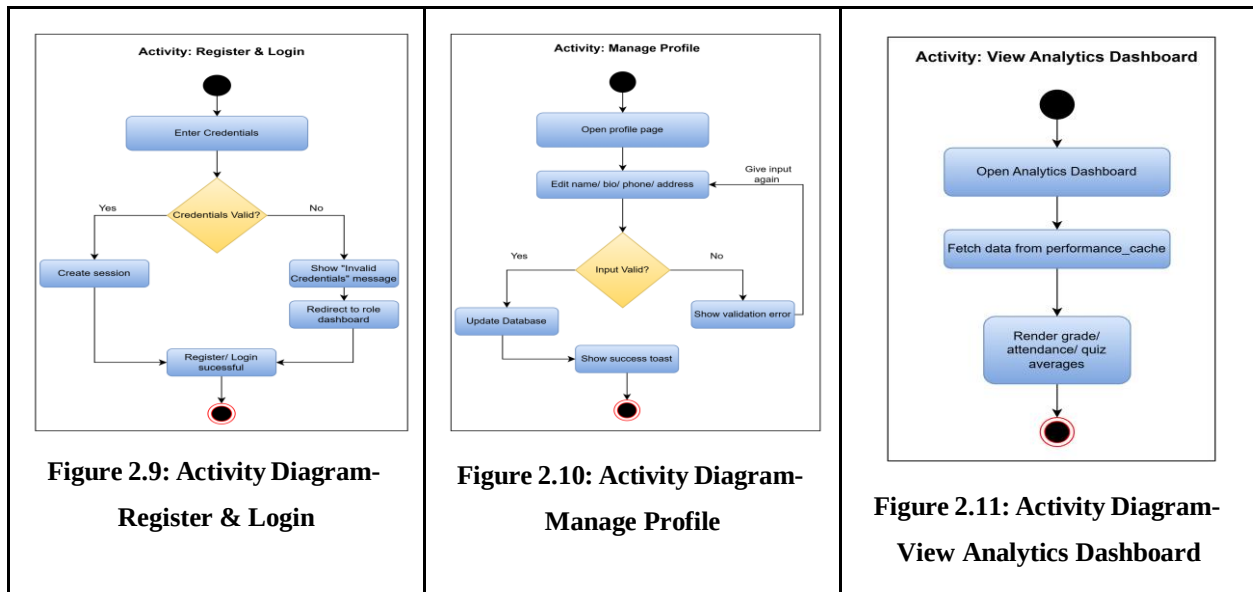


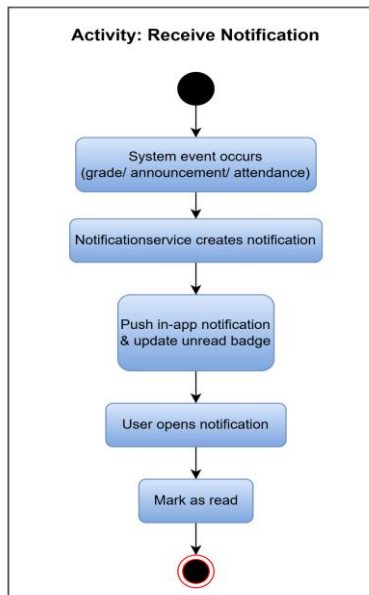
Figure 2.5 : Use Case Diagram for Assessment Engine



2.4.2 Activity Diagrams

Activity diagrams describe the dynamic flow of control within key system processes, including decision points, concurrent activities, and swimlane responsibilities. Two critical workflows are modelled below.





**Figure 2.12: Activity Diagram-
Receive Notification**

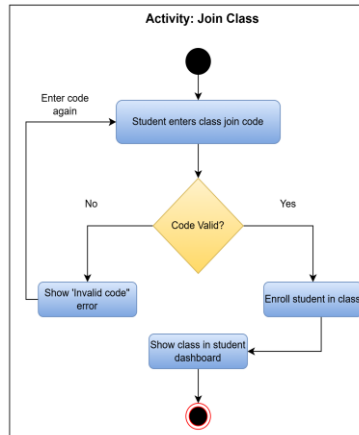
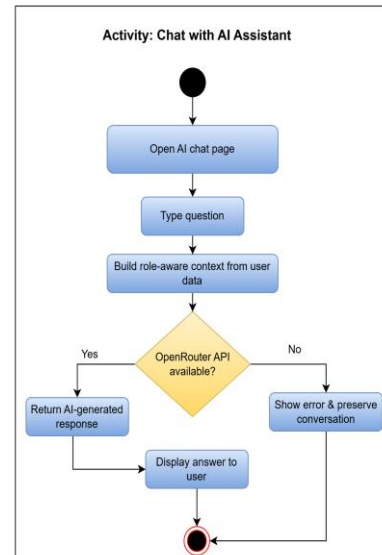


Figure 2.13: Activity Diagram-Join Class



**Figure 2.14: Activity Diagram-
Chat with AI Assistant**

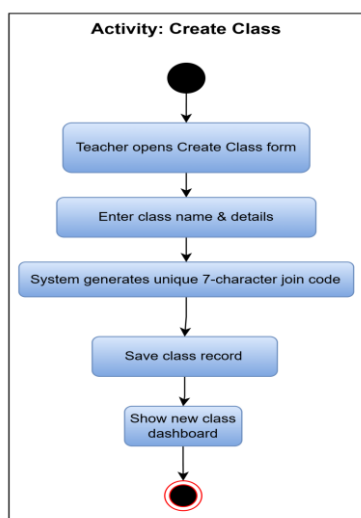
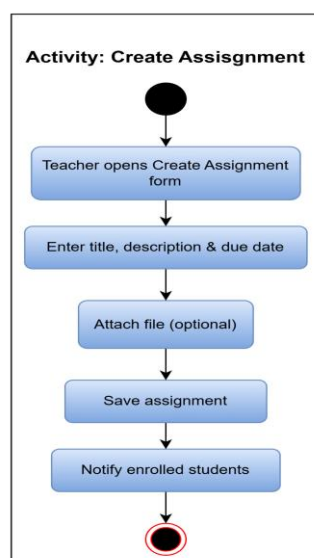


Figure 2.15: Activity Diagram-Create Class



**Figure 2.16: Activity Diagram-
Create Assignment**

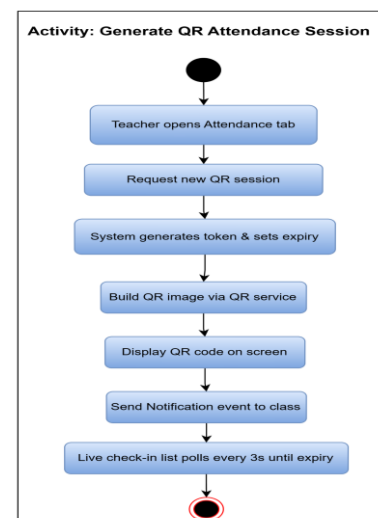


Figure 2.17: Activity Diagram-Generate QR Attendance Session

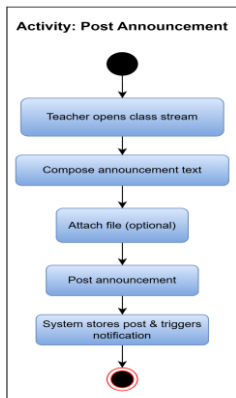


Figure 2.18: Activity Diagram-Post Announcement

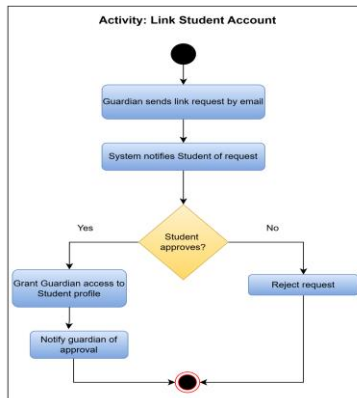


Figure 2.26: Activity Diagram-Link Student Account

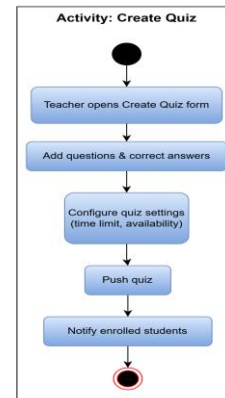


Figure 2.19: Activity Diagram-Create Quiz

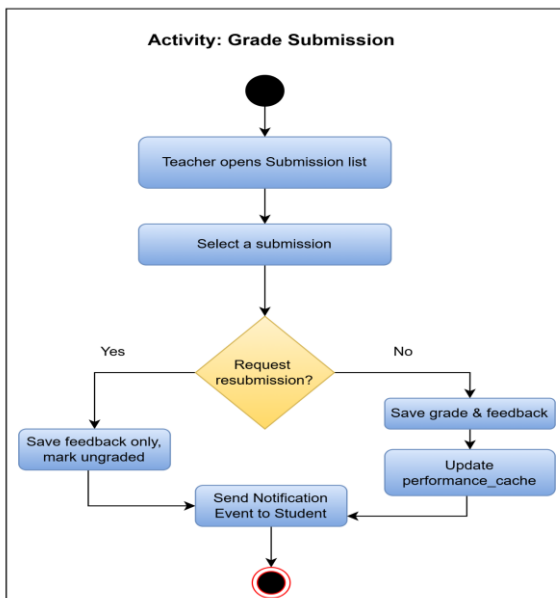


Figure 2.20: Activity Diagram-Grade Submission

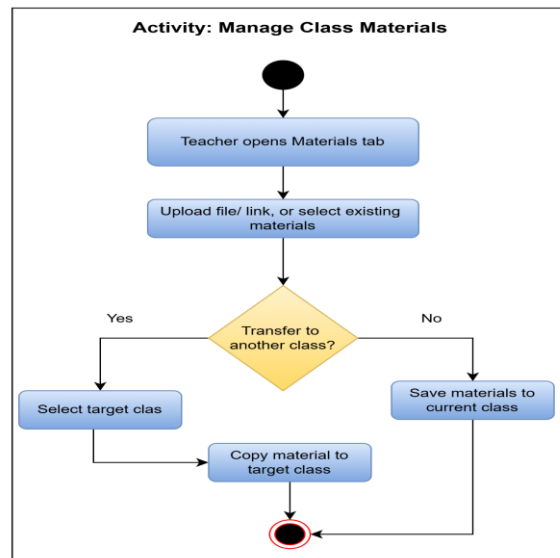


Figure 2.21: Activity Diagram-Manage Class Materials

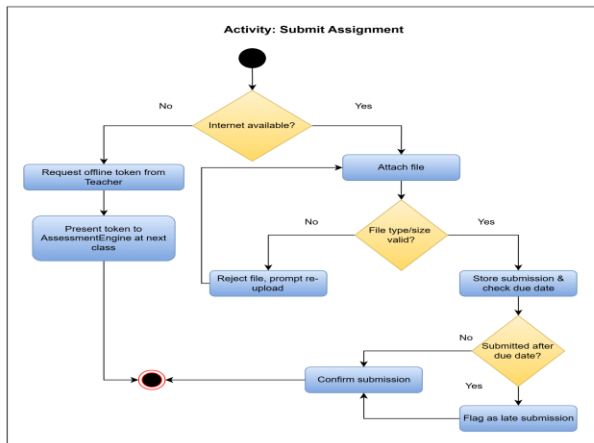


Figure 2.22: Activity Diagram-Submit Assignment

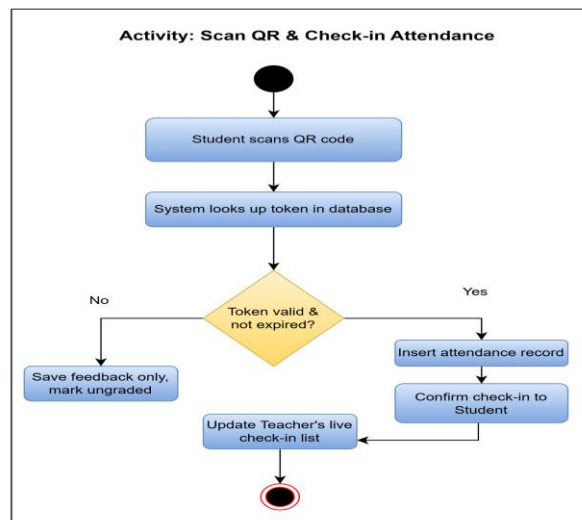


Figure 2.24: Activity Diagram-Scan QR & Check-in Attendance

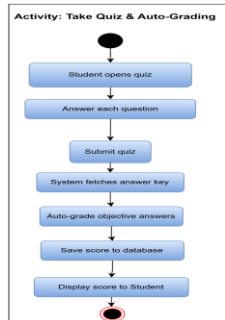


Figure 2.23: Activity Diagram-Take Quiz & Auto-Grading

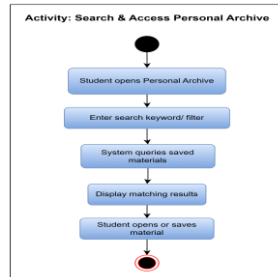


Figure 2.25: Activity Diagram-Search & Access Personal Archive

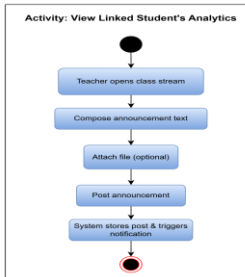


Figure 2.27: Activity Diagram-View Linked Student's Analytics

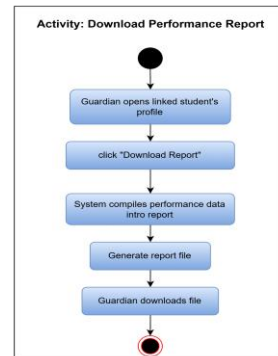


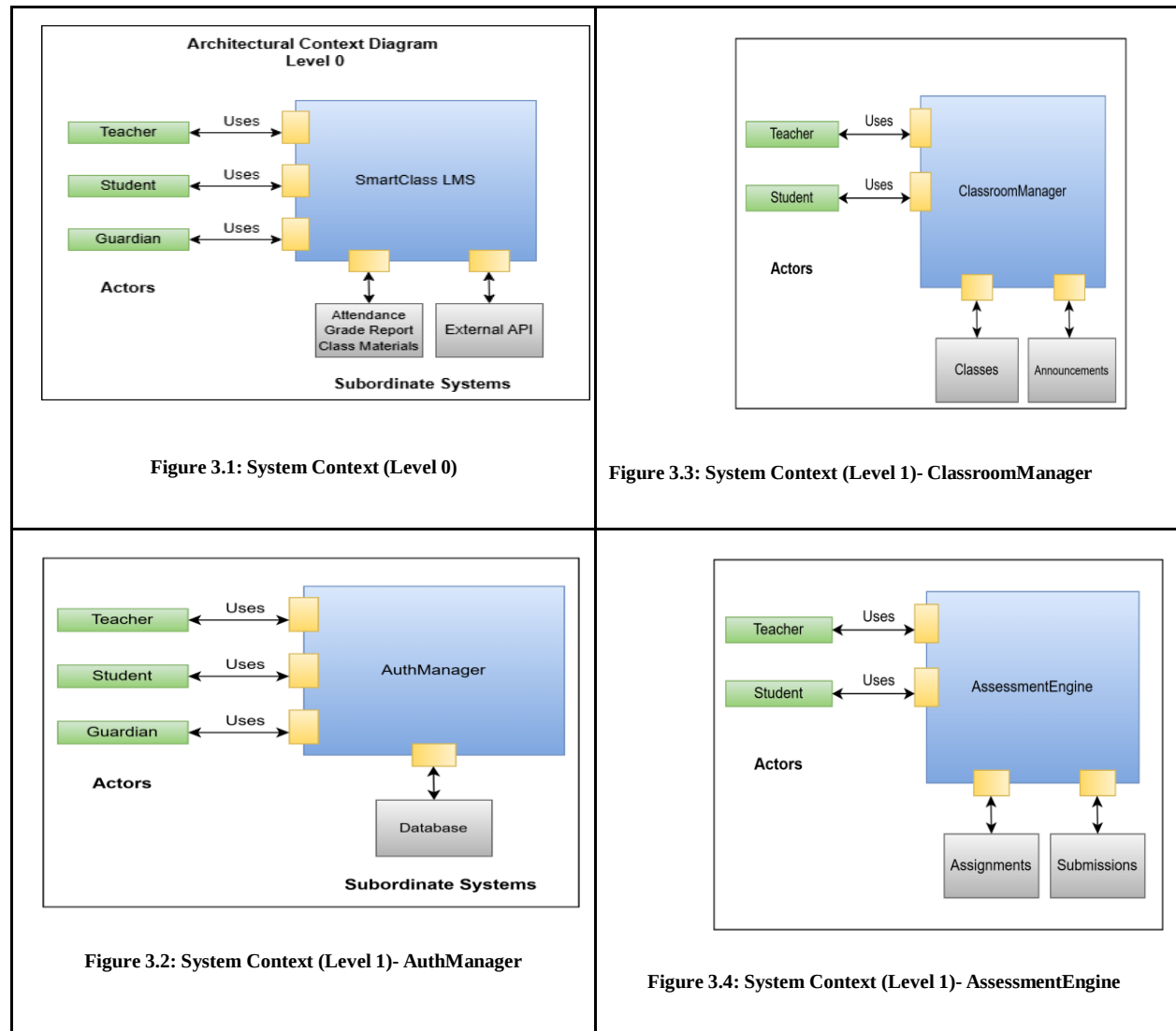
Figure 2.28: Activity Diagram-Download Performance Report

Chapter 3: Software Design

3.1 Architectural Design

SmartClass LMS employs a Layered (N-Tier) Architecture with Server-Side Rendering (SSR) and progressive JavaScript enhancement. This style was selected for four primary reasons: simplicity (no build pipeline is required, as PHP renders complete HTML pages on the server), security (business logic never reaches the client browser), maintainability (clear separation between presentation, logic, and data layers), and accessibility (core features function even when JavaScript is disabled).

3.1.1 Architectural Context Diagram



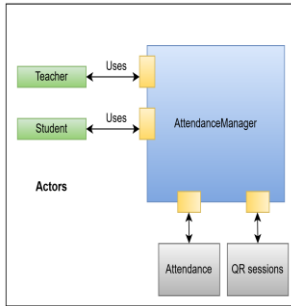


Figure 3.5: System Context (Level 1)- AttendanceManager

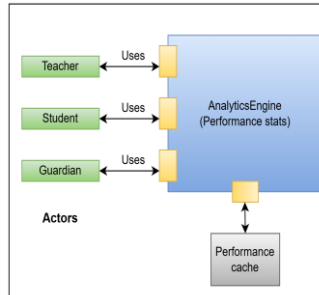


Figure 3.6: System Context (Level 1)- AnalyticsEngine

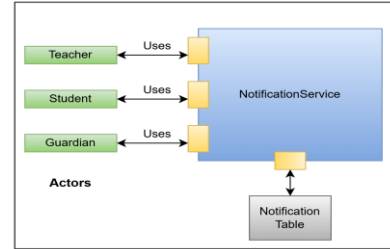


Figure 3.7: System Context (Level 1)- NotificationService

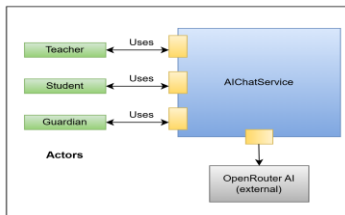


Figure 3.8: System Context (Level 1)- AIChatService

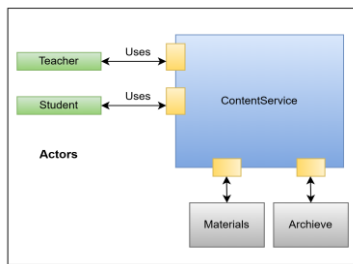


Figure 3.9: System Context (Level 1)- ContentService

3.1.2 Top-Level Component Diagram

The system decomposes into eight high-level components, each encapsulating a cohesive feature domain. Components communicate through the shared DatabaseHelper (PDO) and the NotificationController, which acts as a notification bus receiving calls from four distinct source components.

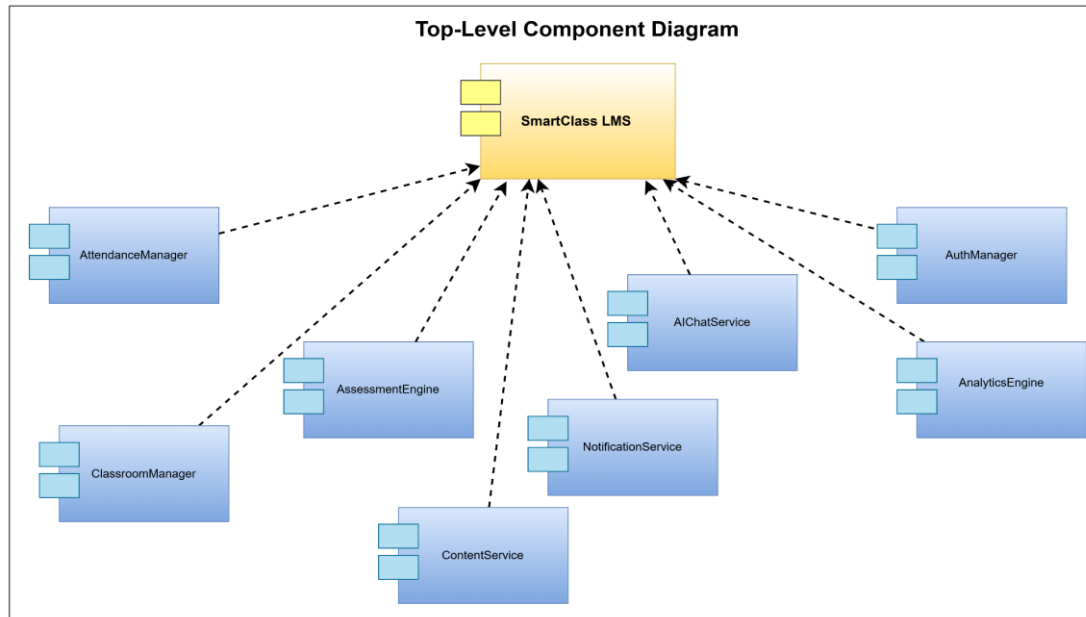


Figure 3.10: Top-Level Components

3.1.3 Instantiation of Each Component with Component Elaboration

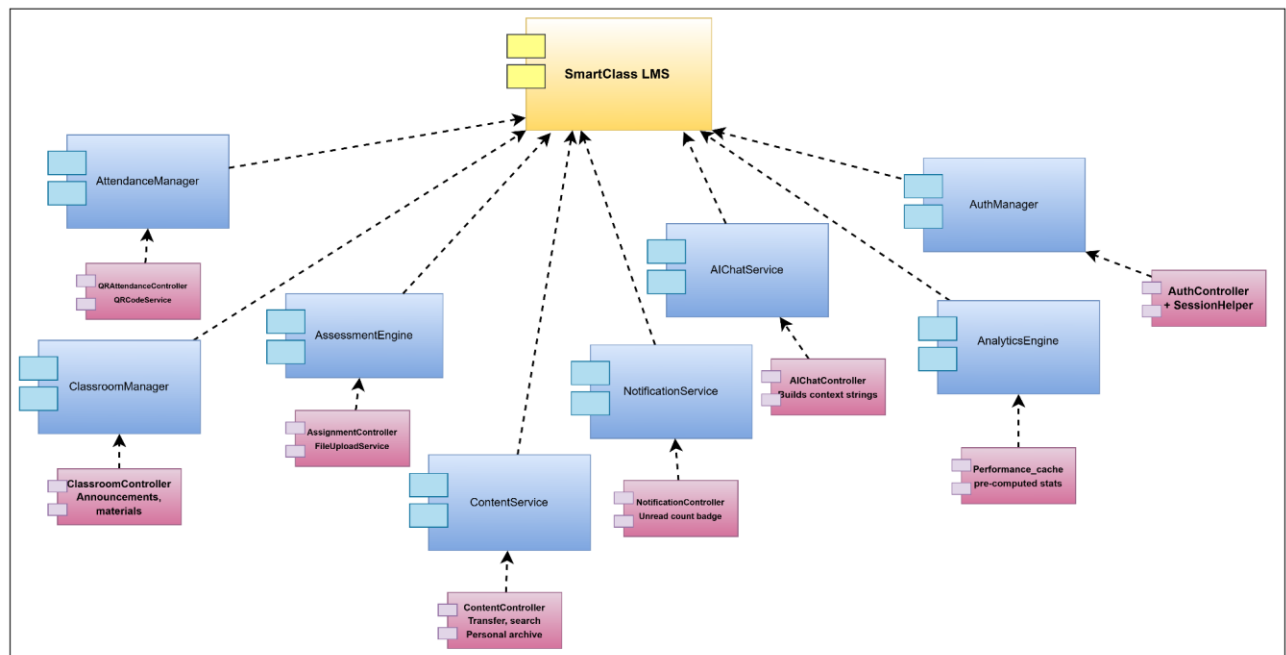


Figure 3.11: Instantiation of Each Component with Component Elaboration

3.2 Component-Level Design

3.2.1 Elaboration of Design Components

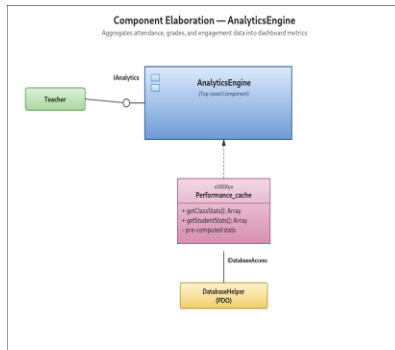


Figure 3.12: Component Elaboration- AnalyticEngine

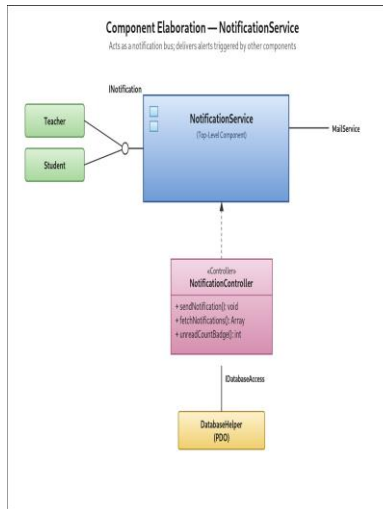


Figure 3.13: Component Elaboration-NotificationService

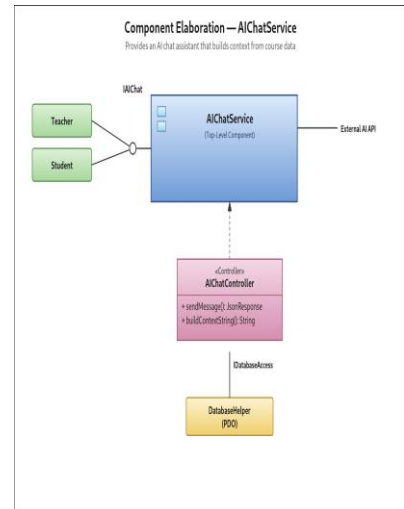


Figure 3.14: Component Elaboration- AIChatService

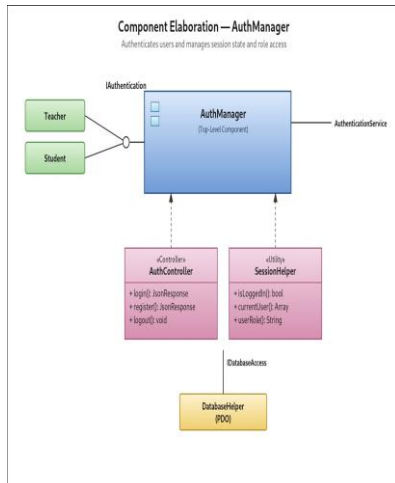


Figure 3.15: Component Elaboration- AuthManager

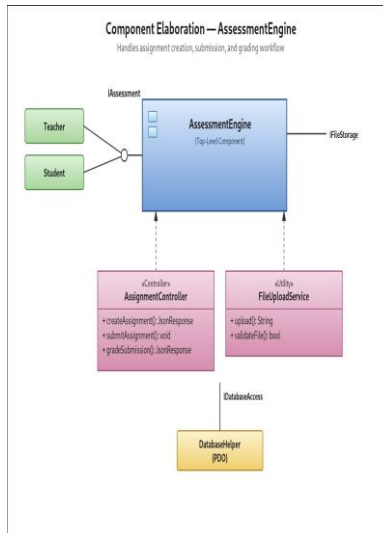


Figure 3.16: Component Elaboration- AssessmentEngine

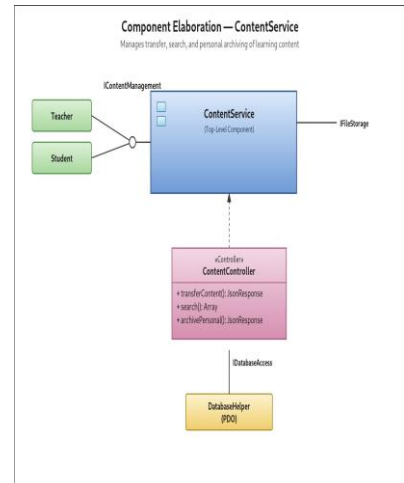


Figure 3.17: Component Elaboration-ContentService

3.2.2 Class Diagram

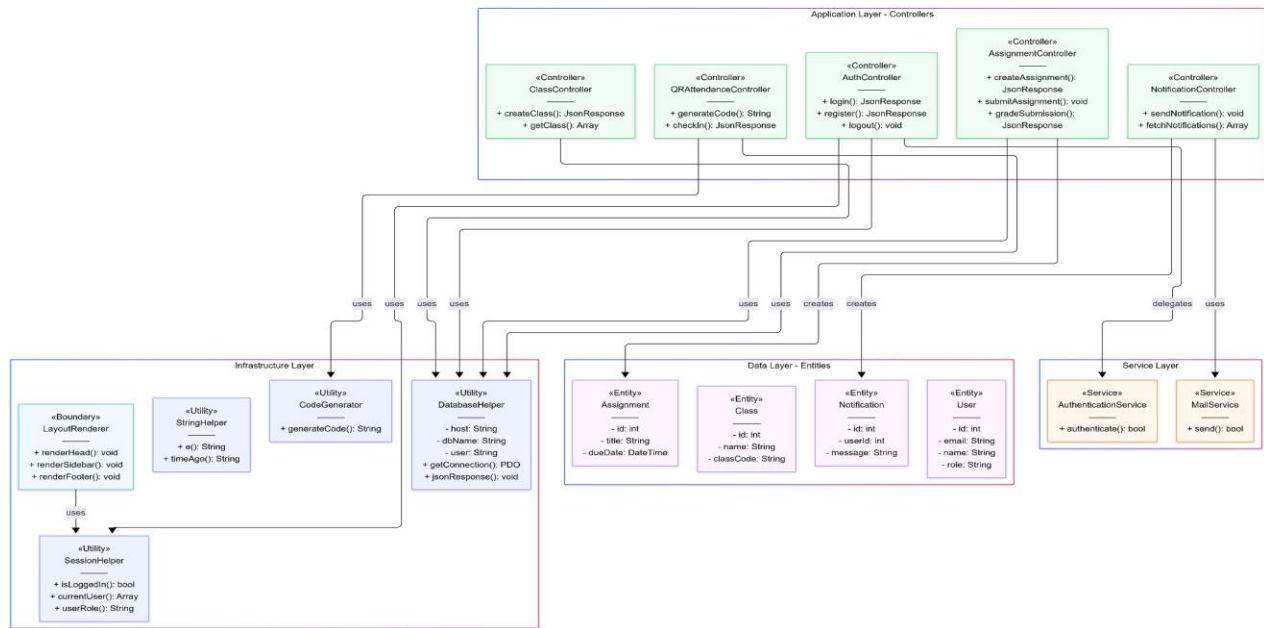


Figure 3.18: Class Diagram

3.2.3 Database Design

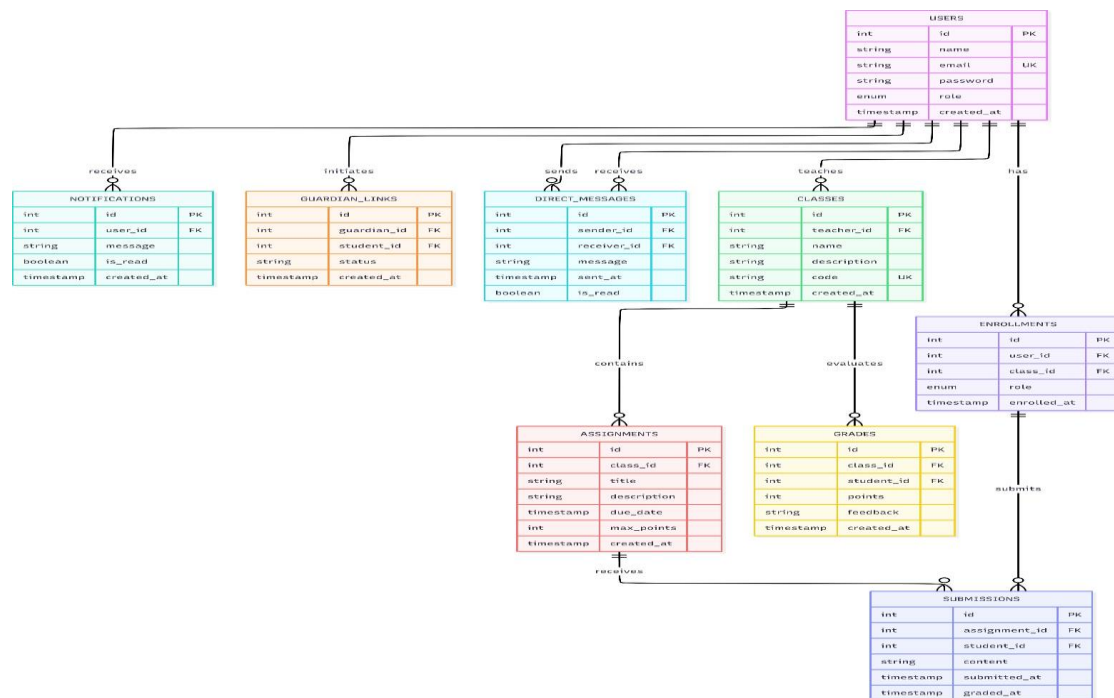
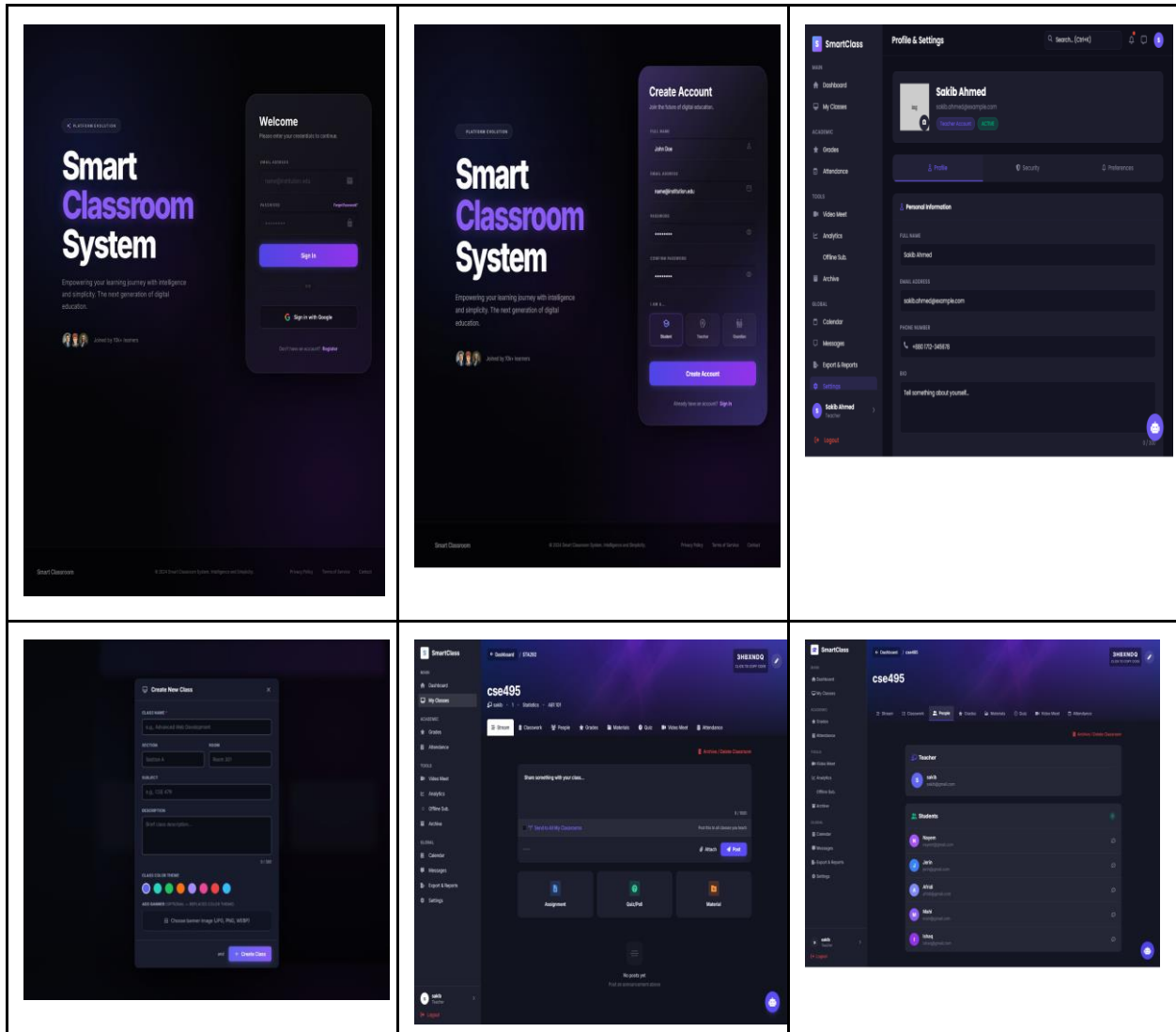
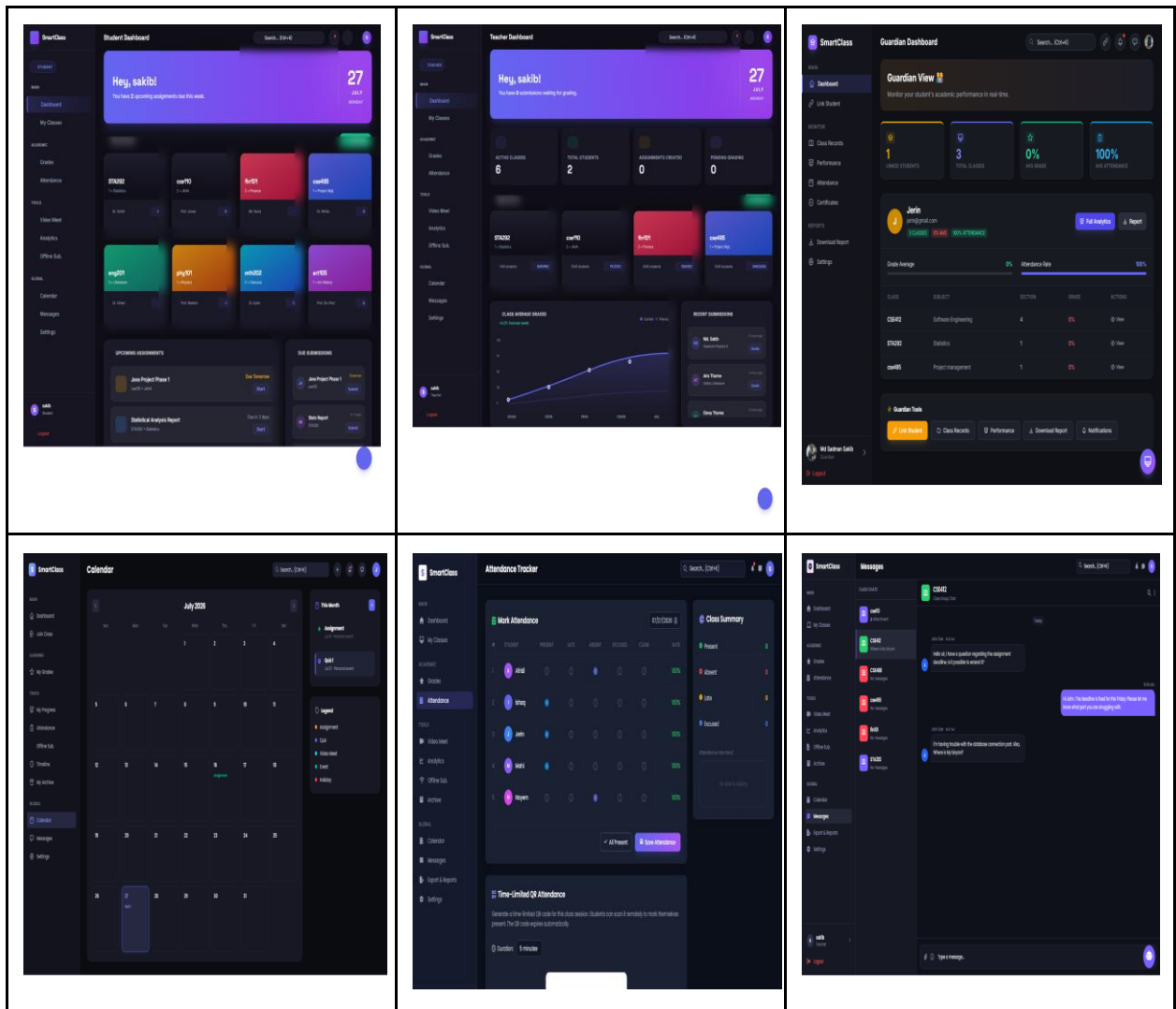


Figure 3.19: Database Design

3.3.2 UI Frameworks





Chapter 4: Implementation

4.1 Code Structure Overview

The SmartClass LMS codebase is organised into a directory hierarchy that directly mirrors the layered architecture described in Chapter 3. Each directory corresponds to a distinct feature domain or system layer, making the project navigable and maintainable without a dependency graph. The following directory tree provides a complete structural overview of the project.

Smart_Class/	← Project root
— index.php	← Login / Register entry page
— login.php	← Login page (redirects from index.php)
— register.php	← User registration page
— config/	
— db.php	← DatabaseHelper: PDO singleton, helpers, constants
— layout.php	← LayoutRenderer: sidebar, topbar, footer partials
— dashboard/	
— teacher.php	← Teacher dashboard
— student.php	← Student dashboard
— guardian.php	← Guardian dashboard
— classroom/	
— index.php	← Classroom hub (Stream, Classwork, People, Grades, Materials)
— attendance.php	← QR Attendance system (teacher + student views)
— quiz.php	← Quiz & Poll system
— grades.php	← Grade management & submission review
— video_meet.php	← Video Meet session management
— archive.php	← Archive & personal storage viewer
— qr_scan.php	← QR code scan landing page
— api/	
— auth.php	← Authentication API (login, register, logout, profile)
— classes.php	← Class CRUD, announcement, material upload
— assignments.php	← Assignment create / submit / grade
— quiz.php	← Quiz & Poll API (create, submit, vote)
— qr_attendance.php	← QR attendance session management
— archive.php	← Archive & personal storage API
— material_transfer.php	← Material import / transfer between classes
— search.php	← Global search suggestions API
— notifications.php	← Notification management API
— ai_chat.php	← AI Chatbot integration (OpenRouter API)
— analytics/	
— teacher_analytics.php	← Teacher-facing analytics page
— performance.php	← Student performance analytics dashboard
— global/	
— calendar.php	← Calendar with events & assignment due dates
— messages.php	← Direct messaging system
— notifications.php	← Notification centre page

—— profile.php	← User profile management
—— export.php	← CSV export for grades
—— guardian/	
—— link.php	← Link student accounts to guardian
—— records.php	← View linked student records
—— download.php	← Download student materials
—— offline/	
—— submission.php	← Offline assignment submission workflow
—— assets/	
—— css/style.css	← Main stylesheet (dark theme, CSS variables)
—— js/main.js	← SCS utility library (apiRequest, showToast, etc.)
—— database/	
—— schema.sql	← Full 22-table database schema
—— update_v4.0.sql	← Migration script for archive and QR tables
—— uploads/	← File upload directory (assignment files, avatars)

Figure 4.1: Directory Structure — SmartClass LMS Project

4.1.1 Key Implementation Decisions

Several implementation decisions were made to balance simplicity, security, and performance within the constraints of a XAMPP-hosted PHP application.

1. **No frontend framework:** The decision to use pure PHP rendering with vanilla JavaScript avoids a Node.js build pipeline, making deployment as simple as copying files to the htdocs directory and importing the SQL schema.
2. **action parameter routing:** Each API file (api/*.php) uses a single entry point with an ?action= query parameter to route requests to the appropriate handler function. This avoids the overhead of a full MVC framework router while maintaining a clean separation of concerns.
3. **SCS JavaScript library:** A small custom library in assets/js/main.js provides SCS.apiRequest() (fetch wrapper with JSON handling), SCS.showToast() (toast notifications), SCS.confirmAction() (confirmation dialogs), and SCS.setLoading() (button loading state). This eliminates repetitive boilerplate code across all 30+ PHP pages.
4. **Performance caching:** The performance_cache table stores pre-computed aggregate values (avg_grade, attendance_rate, quiz_avg) per student-class pair, updated on each data change event. This reduces the analytics page load from a costly multi-table JOIN query to a single indexed lookup.
5. **Soft-delete architecture:** Class archival uses a two-step process — updating the class status column to 'archived' and inserting a record into archived_classes with a 48-hour deletion deadline. This prevents permanent data loss from accidental archival without requiring a separate backup mechanism.

4.2 GitHub Repository

URL:

Chapter 5: Deployment

5.1 Deployment Platform

SmartClass LMS is deployed on a local XAMPP environment which is a cross-platform Apache distribution that bundles Apache 2.4, MySQL 8.x, PHP 8.x, and phpMyAdmin into a single installation package.

5.2 Deployment Process

The following step-by-step deployment procedure details the complete installation and configuration process for SmartClass LMS on a XAMPP server.

Install XAMPP: Install XAMPP 8.0+ and enable `pdo_mysql`. Start **Apache** and **MySQL**.

Copy the project: Place `Smart_Class` in `C:\xampp\htdocs\Smart_Class` and ensure `uploads/` is writable.

Create the database: In phpMyAdmin, create `smart_classroom` using `utf8mb4_unicode_ci`.

Import the schema: Import `database/schema.sql`. If upgrading, also import `database/update_v4.0.sql`.

Configure the database: In `config/db.php`, verify the MySQL host, username, password, and database name.

Set the Base URL: Make sure `BASE_URL` is `http://localhost/Smart_Class`.

Configure the AI chatbot: Add an OpenRouter API key and select the desired model in `api/ai_chat.php`.

Launch the application: Visit `http://localhost/Smart_Class` and log in using the provided demo accounts and password `password123`.

Website URL:

Chapter 6: Conclusion

6.1 Key Learnings

The SmartClass LMS project provided practical experience across the full software development lifecycle:

1. **Requirements Engineering:** User stories, prototype reviews, and interviews helped identify real user needs, such as the offline QR submission feature.
2. **Database Design:** Designing 22 normalised tables with foreign keys and appropriate constraints improved data integrity and consistency.
3. **Security:** BCrypt hashing, PDO prepared statements, and XSS protection were integrated from the beginning, making security part of the system architecture.
4. **Progressive Enhancement:** Core PHP functionality was enhanced with JavaScript, Chart.js, and modals while maintaining functionality when JavaScript is unavailable.
5. **API Design:** A consistent `?action=` routing pattern across 10 API endpoints made the system easier to maintain and extend.
6. **External API Integration:** OpenRouter and QR Server integrations demonstrated the importance of handling third-party service failures gracefully.
7. **SOLID Principles:** Applying principles such as Single Responsibility and Dependency Inversion through services like `FileUploadService` and `QRCodeService` improved code organisation and maintainability.

6.2 Limitations

The current SmartClass LMS has several known limitations:

1. **Session Sharing:** Multiple accounts cannot be used simultaneously in different browser tabs because PHP sessions are shared across the browser.
2. **Polling-Based Updates:** Live features rely on 3-second HTTP polling instead of WebSockets, causing up to 3 seconds of delay and increased server load.
3. **External QR Dependency:** QR generation relies on the external QR Server API, so outages or rate limits may affect attendance functionality.

4. **AI Rate Limits:** The chatbot depends on OpenRouter's free API tier, which may experience rate limits, timeouts, or model availability issues.
5. **No Native Mobile App:** The system is responsive on mobile browsers but does not currently have dedicated Android or iOS applications.
6. **External Video Conferencing:** Video classes depend on Google Meet or Zoom links rather than an integrated conferencing system.
7. **Limited Auto-Grading:** Automatic grading supports only MCQ and True/False questions; written answers require manual teacher evaluation.

6.3 Future Plan

The planned SmartClass LMS enhancements focus on improving **real-time performance, user experience, scalability, and integration:**

1. **High Priority – WebSockets:** Replace 3-second polling with WebSockets for real-time attendance, quizzes, chat, and messaging, reducing server load and latency.
2. **High Priority – WebRTC:** Add built-in video conferencing so users can join classes without relying on Google Meet or Zoom.
3. **Medium Priority – PWA & Mobile App:** Introduce offline caching, home-screen installation, and push notifications, followed later by a React Native or Flutter mobile app.
4. **Medium Priority – Client-Side QR Generation:** Use a JavaScript QR library instead of the external QR Server API to improve reliability and reduce network dependency.
5. **Medium Priority – AI Essay Grading:** Use AI to provide preliminary rubric-based scores and feedback that teachers can review and modify.
6. **Low Priority – Teaching Assistant Role:** Add a limited TA role for managing submissions, announcements, and class materials.
7. **Low Priority – LTI Integration:** Support LTI to connect SmartClass LMS with platforms such as Moodle and Canvas.

